DR. A. Q. KHAN INSTITUTE OF COMPUTER SCIENCES AND INFORMATION TECHNOLOGY

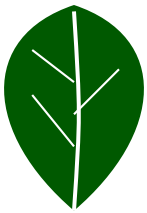
KICSIT KAHUTA

INSTITUTE OF SPACE TECHNOLOGY
KICSIT, Kahuta Campus

MOBILE APPLICATION DEVELOPMENT

ASSIGNMENT NO. 2

THEORY REPORT



LeafGuard AI

AI-Based Plant Leaf Disease Detection Application

Student Name:	Maira Masood
Roll Number:	242201061
Class:	BSCS-5B
Course:	Mobile Application Development
Instructor:	Sir Uzair
Institute:	KICSIT Kahuta
Submission Date:	05 November 2026

Submitted to: Sir Uzair

Contents

1 Introduction **3**

2 Objectives **3**

3 Machine Learning and Deep Learning in Android **3**

3.1 What is Machine Learning? 3

3.2 What is Deep Learning? 3

3.3 Embedding ML/DL Models in Android 4

4 Real-World Applications **4**

5 Benefits of On-Device Machine Learning **4**

6 Challenges of On-Device ML/DL **5**

7 TensorFlow Lite **5**

7.1 What is TensorFlow Lite? 5

7.2 Difference Between TensorFlow and TensorFlow Lite 5

7.3 Why Use TensorFlow Lite in Android? 5

8 Steps to Integrate a TFLite Model into Android **6**

9 Role of TensorFlow Lite Interpreter **6**

10 Reference Repository Analysis **7**

11 Our Project: LeafGuard AI **7**

11.1 Main Features 7

11.2 Working of the Application 8

11.3 Model Information 8

12 Testing and Limitations **8**

13 Conclusion **9**

14 References **9**

1. Introduction

Artificial Intelligence (AI) is becoming an important part of modern mobile applications. It allows mobile applications to perform intelligent tasks such as image recognition, prediction, and classification.

This assignment focuses on integrating Machine Learning (ML) and Deep Learning (DL) models into Android applications using TensorFlow Lite.

Our project is called **LeafGuard AI**. It is a plant leaf disease detection application. The main purpose of this application is to analyze a plant leaf image and predict its possible disease using an AI model.

2. Objectives

The main objectives of this assignment are:

- To understand how ML and DL models work in Android.
- To study TensorFlow Lite.
- To explore a reference Android application.
- To develop an AI-based Android application.
- To understand on-device model prediction.
- To learn how to use a trained model in a mobile application.

3. Machine Learning and Deep Learning in Android

3.1 What is Machine Learning?

Machine Learning is a branch of Artificial Intelligence that allows computers to learn patterns from data and make predictions without being explicitly programmed for every task.

Example: Predicting whether a plant leaf is healthy or diseased.

3.2 What is Deep Learning?

Deep Learning is a part of Machine Learning that uses artificial neural networks with multiple layers to learn complex patterns.

Example: Recognizing plant diseases from leaf images.

3.3 Embedding ML/DL Models in Android

Embedding a model means adding a trained machine learning model into an Android application so that the application can make predictions.

The model receives input data, processes it, and produces an output.

4. Real-World Applications

Machine Learning and Deep Learning are used in many mobile applications.

- Face recognition applications.
- Voice assistants.
- Plant disease detection.
- Object detection.
- Language translation.
- Medical image analysis.
- Recommendation systems.

5. Benefits of On-Device Machine Learning

1. **Low Latency:** Predictions can be made quickly because data does not need to travel to a server.
2. **Offline Working:** The application can work without an internet connection if the model is stored on the device.
3. **Privacy:** User data can remain on the mobile device.
4. **Reduced Server Cost:** No continuous cloud processing is required for every prediction.
5. **Fast Response:** Results can be displayed directly inside the application.

6. Challenges of On-Device ML/DL

- **Limited Memory:** Mobile devices have limited RAM and storage.
- **Battery Usage:** Complex models may consume more battery.
- **Processing Power:** Some devices have slower processors.
- **Model Size:** Large models require more storage.
- **Device Compatibility:** Different Android devices may have different capabilities.

7. TensorFlow Lite

7.1 What is TensorFlow Lite?

TensorFlow Lite is a lightweight framework used to run machine learning models on mobile and edge devices.

It allows developers to use trained models inside Android and iOS applications.

7.2 Difference Between TensorFlow and TensorFlow Lite

TensorFlow	**TensorFlow Lite**
Used for training and developing models.	Mainly used for running trained models.
Can require more resources.	Designed for limited-resource devices.
Commonly used on computers and servers.	Commonly used on mobile and edge devices.
Supports model development.	Supports efficient on-device inference.

7.3 Why Use TensorFlow Lite in Android?

TensorFlow Lite is useful in Android applications because it:

- Supports on-device prediction.
- Provides fast model execution.

- Can work offline.
- Reduces the need for server communication.
- Supports lightweight model formats.

8. Steps to Integrate a TFLite Model into Android

1. **Train the Model:** First, train a machine learning model using a suitable dataset.
2. **Convert the Model:** Convert the trained TensorFlow model into the `.tflite` format.
3. **Create Android Project:** Open Android Studio and create an Android application.
4. **Add Dependency:** Add the TensorFlow Lite library to the project.
5. **Add Model:** Place the `.tflite` model inside the Android assets folder.
6. **Load Model:** Load the model using the TensorFlow Lite Interpreter.
7. **Prepare Input:** Convert the input image into the format expected by the model.
8. **Run Inference:** Pass the input to the model and get its output.
9. **Display Result:** Show the predicted result on the application screen.

9. Role of TensorFlow Lite Interpreter

The Interpreter is responsible for executing a TensorFlow Lite model.

Its main functions are:

- Loading the model.
- Receiving input data.
- Running model operations.
- Producing output predictions.

10. Reference Repository Analysis

The reference repository provided for this assignment is:

`https://github.com/Uzair390-Del/NumberGuessingAppTflite`

The purpose of studying the reference project is to understand how a TensorFlow Lite model is integrated into an Android application.

The following points should be examined in the repository:

- Android project structure.
- Gradle dependencies.
- Model file and assets.
- User interface design.
- Model loading process.
- Input and output handling.
- Prediction display.

Note: The actual repository files and code should be checked in Android Studio before documenting specific implementation details.

11. Our Project: LeafGuard AI

LeafGuard AI is an AI-based plant leaf disease detection application.

Its purpose is to analyze a plant leaf image and classify it using a trained deep learning model.

11.1 Main Features

- Splash screen.
- Login screen.
- Home and leaf detection screen.
- Leaf image selection.
- AI-based image classification.
- Display of predicted class.

11.2 Working of the Application

The application follows these basic steps:

1. The user opens LeafGuard AI.
2. The splash screen appears.
3. The user reaches the login screen.
4. The user selects a plant leaf image.
5. The image is prepared for the AI model.
6. The TensorFlow Lite model processes the image.
7. The application interprets the output.
8. The predicted class is displayed to the user.

11.3 Model Information

The model used in the project has the following reported input and output shapes:

- Input shape: `[1, 224, 224, 3]`
- Input data type: `float32`
- Output shape: `[1, 38]`
- Output data type: `float32`

The input represents one RGB image of size 224 by 224 pixels. The output contains 38 class scores.

The exact class names and preprocessing method must match the model used during training.

12. Testing and Limitations

Testing is necessary to check whether an application works correctly.

The following areas should be tested:

- Application launch.

- Navigation between screens.
- Image selection.
- Image resizing and preprocessing.
- Model loading.
- Model prediction.
- Display of the correct class label.

A successful screen display does not automatically prove that AI inference is working. Actual model predictions should be tested using sample images and recorded as evidence.

13. Conclusion

This assignment explains how machine learning and deep learning models can be integrated into Android applications using TensorFlow Lite.

LeafGuard AI demonstrates the idea of using an image classification model for plant leaf disease detection. It helps us understand model input, model output, preprocessing, inference, and result display.

Through this assignment, we gain practical knowledge about AI-based mobile application development.

14. References

1. TensorFlow Lite Documentation: `https://www.tensorflow.org/lite`
2. Android Developers: `https://developer.android.com/`
3. TensorFlow Documentation: `https://www.tensorflow.org/`
4. Reference Repository: `https://github.com/Uzair390-Del/NumberGuessingAppTflite`
5. Project Repository: `https://github.com/maira123669/LeafGuard.AI`